UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS

NOMBRE DEL CURSO

PRÁCTICA SEMANA 6: FRAMEWORK JS: EVENTOS, RENDERIZADO CONDICIONAL/ITERATIVO, FORMULARIOS, ROUTING Y CONSUMO DE APIS (ASYNC/AWAIT, AXIOS)

PRESENTADO POR:

CHAHUAYO MARTINEZ JUAN CARLOS

DOCENTE:

AGUILAR CORONACIÓN MIGUEL ELIAS

HUANCAYO - 2026

1. **Link del proyecto**

https://github.com/devjuancarlos10/practica-semana6.git

2. **Tecnologías utilizadas**

Tecnología	Uso en el proyecto
React 18	Construcción de la interfaz basada en componentes
Vite	Entorno de desarrollo y empaquetado rápido
React Router DOM	Enrutamiento del lado del cliente
Axios	Cliente HTTP para consumo de la API REST
Context API	Estado global compartido (`AppContext`)
CSS (tema blanco médico)	Estilos, animaciones y microinteracciones

3. **Configuración del proyecto**

Se crea el proyecto con Vite, que ofrece un servidor de desarrollo con recarga instantánea (HMR) y una configuración mínima para React. Luego se instalan las dependencias necesarias y se organiza el código en carpetas por responsabilidad.

Comandos utilizados

```
# Crear el proyecto con la plantilla de React
npm create vite@latest vetcare-plus -- --template react


# Entrar a la carpeta e instalar dependencias base
cd vetcare-plus
npm install


# Instalar las dependencias adicionales de la práctica
npm install react-router-dom axios


# Levantar el servidor de desarrollo
npm run dev
```

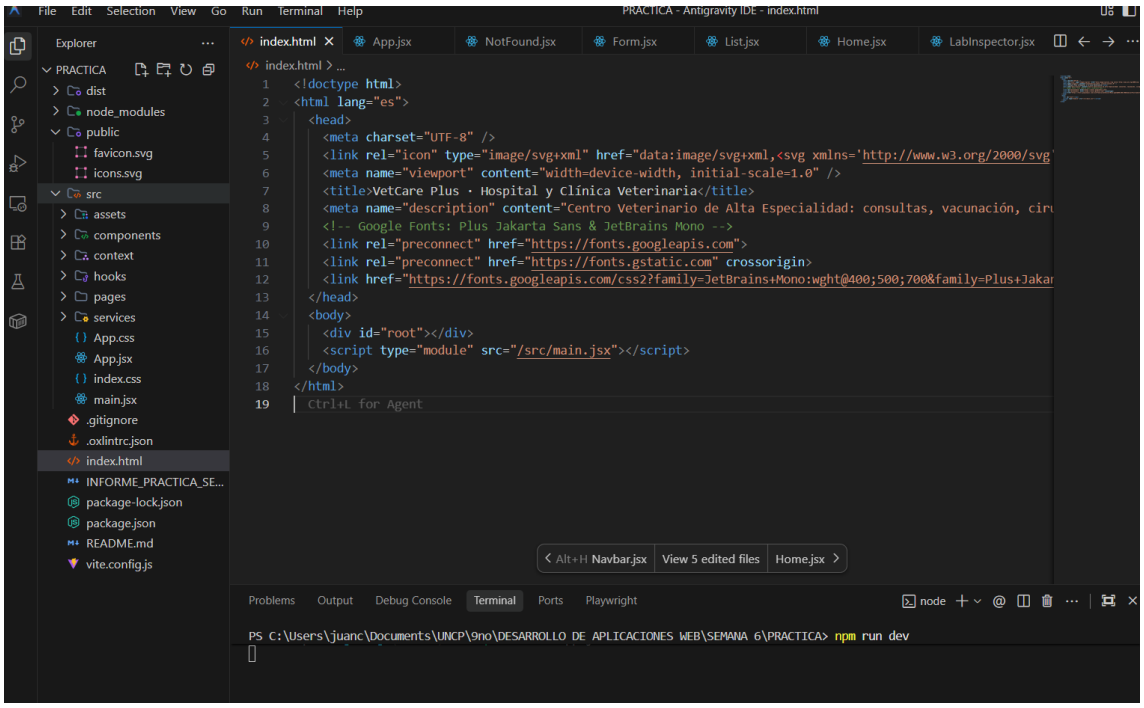
La aplicación queda disponible en `http://localhost:5174/` (el puerto puede variar si el 5173 está ocupado).

Árbol de carpetas

```
vetcare-plus/
├── public/
├── src/
│   ├── components/   # Componentes reutilizables (Navbar, LabInspector, etc.)
│   ├── context/      # Estado global (AppContext.jsx)
│   ├── hooks/        # Hooks personalizados (useUsers.js)
│   ├── pages/        # Páginas de cada ruta (Home, Users, Form, NotFound)
│   ├── App.jsx       # Definición de rutas
│   ├── main.jsx      # Punto de entrada de React
│   └── index.css     # Estilos globales
├── index.html
├── package.json
└── vite.config.js
```

Justificación de la estructura

- **components/**: piezas de interfaz reutilizables y sin lógica de ruta.
- **pages/**: una página por ruta, que compone componentes.
- **hooks/**: lógica de datos separada de la vista, lo que facilita su reutilización y prueba.
- **context/**: estado compartido entre páginas sin pasar props manualmente (prop drilling).



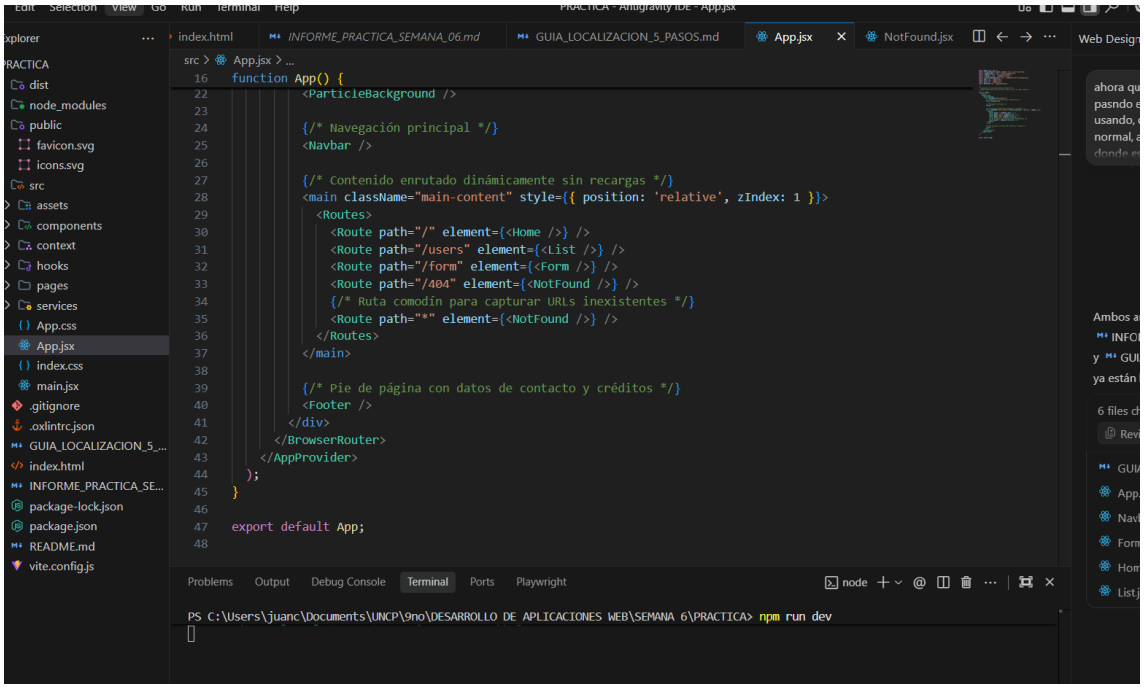
4. Enrutamiento con browerrouter y navlink

Se implementa una **SPA (Single Page Application)**: el navegador carga una sola vez el HTML y React Router cambia la vista según la URL, sin recargar la página. `BrowserRouter` habilita el historial del navegador, `Routes` y `Route` asocian rutas con páginas, y `NavLink` permite resaltar el enlace activo.

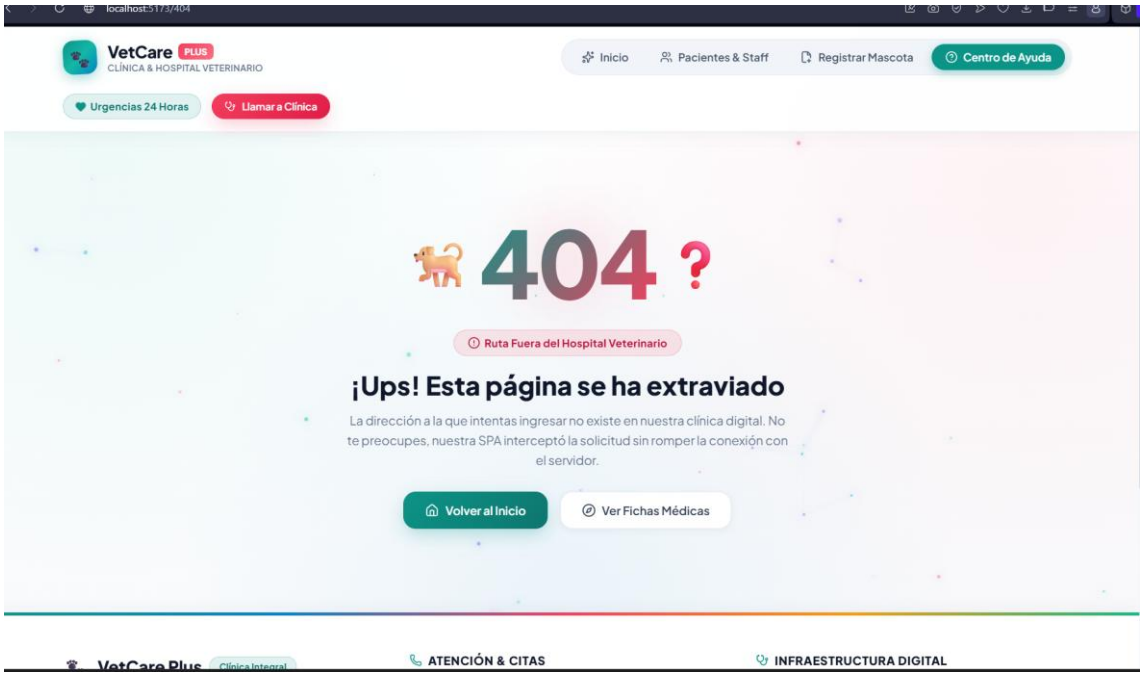
Configuración de rutas (src/App.jsx)

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import Navbar from './components/Navbar';
import Home from './pages/Home';
import Users from './pages/Users';
import Form from './pages/Form';
import NotFound from './pages/NotFound';

export default function App() {
  return (
    <BrowserRouter>
      <Navbar />
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/usuarios" element={<Users />} />
        <Route path="/formulario" element={<Form />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}
```

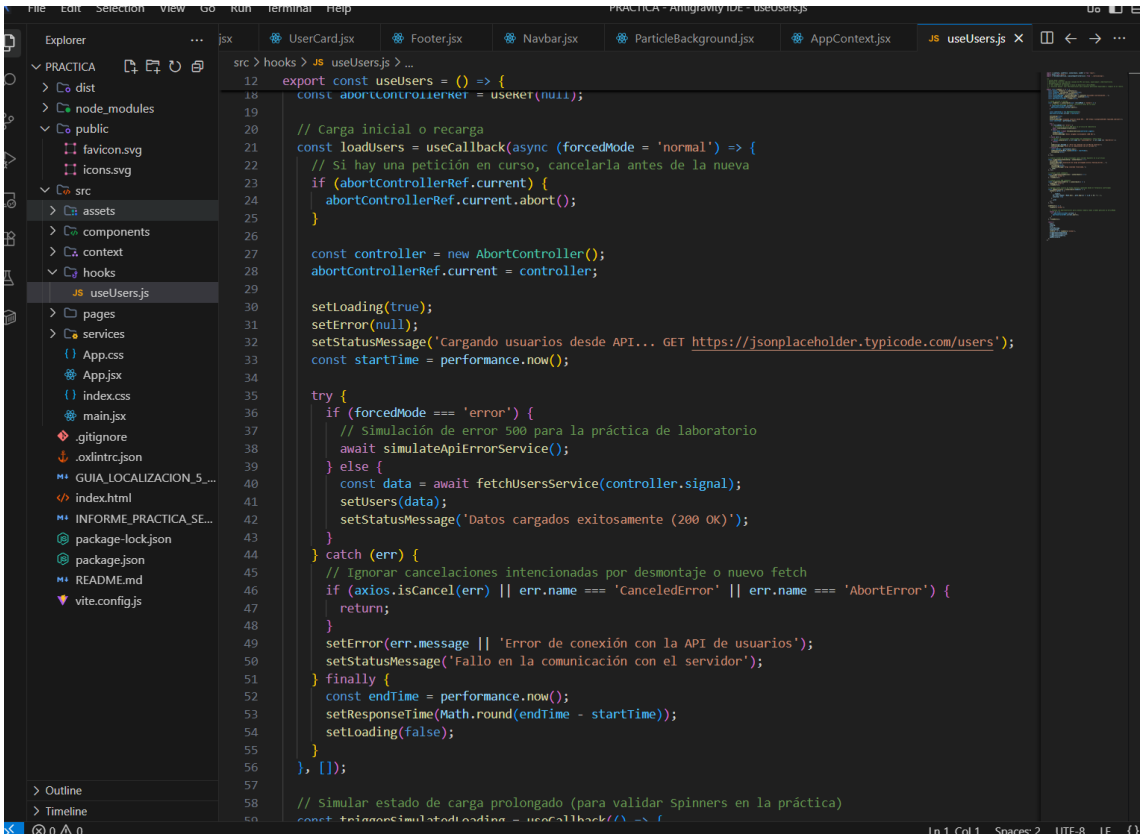


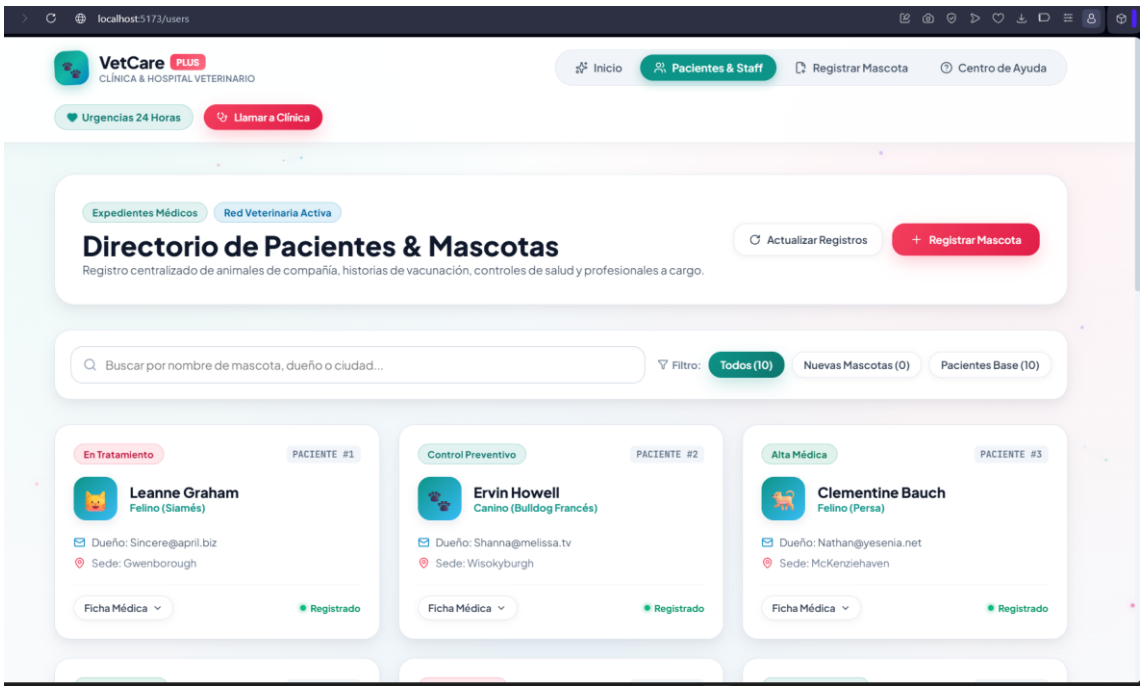
4.


localhost:5173/404
VetCare PLUS
CLÍNICA & HOSPITAL VETERINARIO
Inicio
Pacientes & Staff
Registrar Mascota
Centro de Ayuda
Urgencias 24 Horas
Llamar a Clínica
404 ?
Ruta Fuera del Hospital Veterinario
¡Ups! Esta página se ha extraviado
La dirección a la que intentas ingresar no existe en nuestra clínica digital. No te preocupes, nuestra SPA interceptó la solicitud sin romper la conexión con el servidor.
Volver al Inicio
Ver Fichas Médicas
VetCare Plus
Clínica Integral
ATENCIÓN & CITAS
INFRAESTRUCTURA DIGITAL

4. CONSUMO DE API CON AXIOS, ASYNC/AWAIT Y ABORTCONTROLLER

Se consume una API REST usando **Axios** dentro de un hook personalizado (`useUsers`). El hook gestiona tres estados (`loading`, `error`, `data`) y usa `AbortController` para cancelar la petición si el componente se desmonta antes de recibir la respuesta, evitando actualizaciones de estado sobre componentes que ya no existen y fugas de memoria.





6. Renderizado con .map() y key, y formulario controlado

Se renderizan colecciones de datos con `.map()`, asignando una `key` única y estable a cada elemento para que React identifique qué ítems cambian, se añaden o se eliminan, sin generar warnings en consola. Además, se construye un formulario cuyos campos están **controlados** por `useState`: el valor de cada input proviene del estado y se actualiza con `onChange`.

```
<ul className="user-list">
  {users.map((user) => (
    <li key={user.id} className="user-card">
      <h3>{user.name}</h3>
      <p>{user.email}</p>
    </li>
  ))}
</ul>
```

Formulario controlado (src/pages/Form.jsx)

```
import { useState } from 'react';


export default function Form() {
  const [form, setForm] = useState({ nombre: '', especie: '', edad: '' });


  const handleChange = (e) => {
    const { name, value } = e.target;
    setForm((prev) => ({ ...prev, [name]: value }));
  };


  const handleSubmit = (e) => {
    e.preventDefault();
    console.log('Datos enviados:', form);
  };


  return (
    <form onSubmit={handleSubmit}>
      <input name="nombre" value={form.nombre} onChange={handleChange}
placeholder="Nombre" />
      <input name="especie" value={form.especie} onChange={handleChange}
placeholder="Especie" />
```

```
      <input name="edad" type="number" value={form.edad} onChange={handleChange}
placeholder="Edad" />

      <button type="submit">Guardar</button>



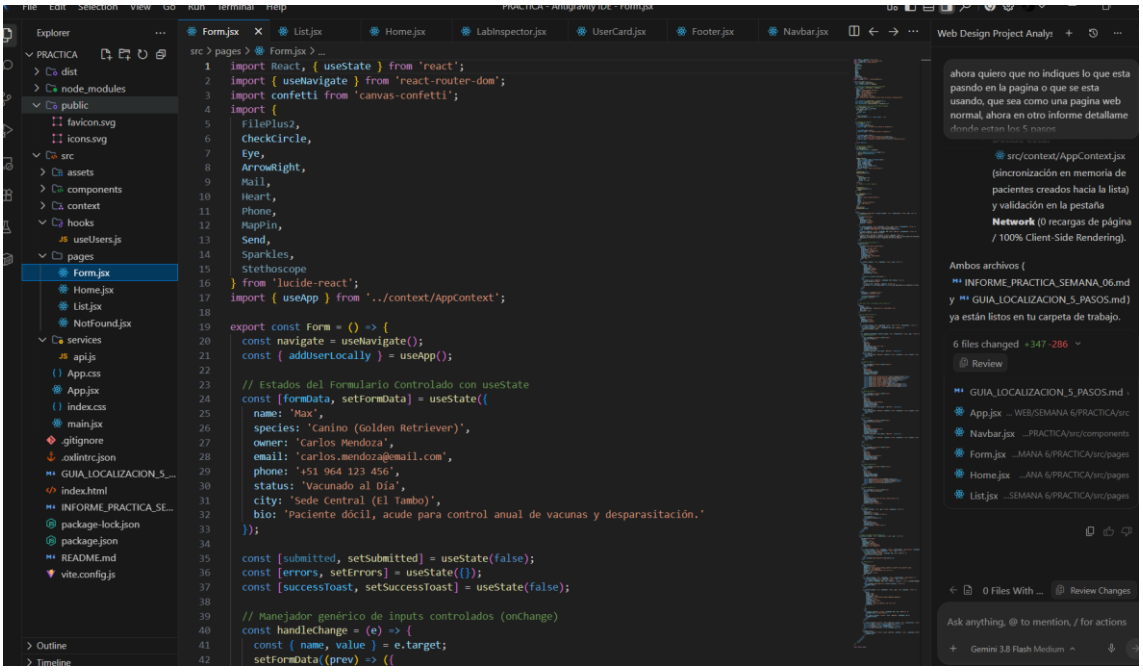
      {/* Vista previa reactiva del estado */}

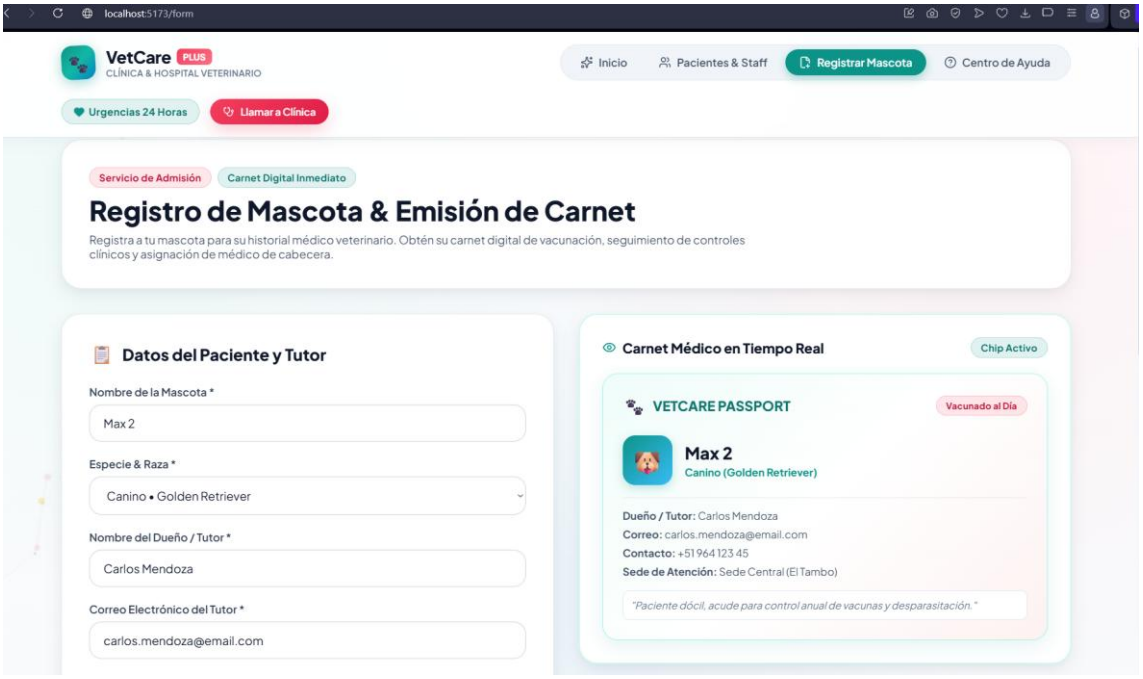
      <pre>{JSON.stringify(form, null, 2)}</pre>

    </form>

  );

}
```





7. PASO 5: AUDITORÍA CSR, DEVTOOLS Y SINCRONIZACIÓN GLOBAL

